



REPORTE TÉCNICO

# Pipeline de observabilidad end-to-end con OpenTelemetry

Dos microservicios · Prometheus · Jaeger · Grafana · Loki

Realizado por:

Yerlinson Maturana Serna  
Oscar Eduardo Clavijo Alarcón  
Camilo Andrés Porras Martín  
Edward Daniel Porras Martín

| Entregable              | Estado                      |
|-------------------------|-----------------------------|
| SDK OTEL y tres señales | Implementado y probado      |
| Collector local/GCP/AWS | Versionado y validado       |
| Dashboard Grafana       | 6 paneles con datos         |
| IaC                     | Helm y Terraform válidos    |
| Benchmark               | Mediciones reales incluidas |

## 1. Objetivo y alcance

El sistema captura trazas distribuidas, métricas y logs estructurados de dos microservicios. service-a recibe una orden, valida la entrada y llama por HTTP a service-b. service-b calcula el precio y realiza escritura y lectura en SQLite. El contexto W3C traceparent viaja en la llamada HTTP, por lo que los spans de ambos servicios forman una sola traza.

### Flujo de señales

| Señal    | Origen                                | Ruta                                      | Uso                     |
|----------|---------------------------------------|-------------------------------------------|-------------------------|
| Trazas   | FastAPI, HTTPX, SQLite y spans custom | OTLP → Collector → Jaeger/X-Ray           | Camino de una solicitud |
| Métricas | SDK OTel y runtime                    | Endpoint → Prometheus                     | SLIs y capacidad        |
| Logs     | logging JSON + OTLP                   | Collector → Loki/Cloud Logging/CloudWatch | Eventos con trace_id    |

### Criterios de éxito

Una solicitud debe mostrar spans padre-hijo en service-a y service-b, un span de base de datos y el mismo trace\_id en los logs. Prometheus debe marcar todos los targets como UP; el dashboard debe mostrar tráfico, p99, errores, disponibilidad, CPU y errores del Collector.

## 2. Instrumentación con OpenTelemetry

La aplicación usa FastAPI y el SDK de Python. FastAPIInstrumentor crea spans de servidor; HTTPXClientInstrumentor crea spans de cliente e inyecta Trace Context; el instrumentador SQLite registra operaciones de base de datos mediante cursores instrumentados. Los spans validate-order y calculate-price representan lógica crítica.

### Diseño de logs

TraceJsonFormatter serializa timestamp, severidad, mensaje, service.name, deployment.environment, trace\_id y span\_id. El identificador se toma del span activo y se emite por stdout y OTLP. Esta estrategia permite usar trace\_id como pivote desde Grafana Logs hacia Jaeger.

### Métricas y SLIs

| Panel          | Consulta conceptual             | Interpretación          |
|----------------|---------------------------------|-------------------------|
| Throughput     | rate(request_count)             | Solicitudes por segundo |
| Latencia p99   | histogram_quantile(0.99, ...)   | Cola larga percibida    |
| Errores        | 5xx / total                     | Confiabilidad           |
| Disponibilidad | avg(up)                         | Targets accesibles      |
| CPU            | rate(process_cpu_seconds_total) | Costo de cómputo        |
| Collector      | send_failed_*                   | Pérdida de telemetría   |

Se evita colocar order\_id como etiqueta de métrica porque produciría alta cardinalidad; sí es apropiado como atributo de span.

### 3. Collector y despliegues

Los tres pipelines aplican `memory_limiter` antes de batch para proteger el proceso ante picos. `resource` normaliza atributos de plataforma y ambiente. `batch` reduce llamadas a los backends. OTLP se habilita por gRPC 4317 y HTTP 4318.

| Entorno | Trazas               | Métricas                      | Logs            |
|---------|----------------------|-------------------------------|-----------------|
| Local   | Jaeger por OTLP      | Prometheus                    | Loki            |
| GCP     | Jaeger + Cloud Trace | Prometheus + Cloud Monitoring | Cloud Logging   |
| AWS     | AWS X-Ray            | Prometheus                    | CloudWatch Logs |

#### GKE

El chart crea namespace, Collector replicado, servicios y deployments. Workload Identity debe vincular la ServiceAccount de Kubernetes con una cuenta GCP con permisos mínimos de escritura de trazas, métricas y logs.

#### ECS Fargate

La tarea agrupa `service-a`, `service-b` y AWS Distro for OpenTelemetry como sidecar; localhost simplifica OTLP. La definición incluye roles para X-Ray y CloudWatch y carga la configuración versionada del Collector.

#### Seguridad

No se codifican credenciales. En cloud se usan identidades de workload, red privada, retención limitada y control de atributos. No deben registrarse tokens, datos personales ni cuerpos completos.

### 4. Correlación y operación

El procedimiento de diagnóstico parte de un SLI degradado, selecciona una ventana temporal, abre una traza lenta, identifica el span dominante y busca su `trace_id` en logs. Esta navegación conecta el síntoma cuantitativo con la ejecución concreta.

| Prueba                         | Resultado            | Evidencia             |
|--------------------------------|----------------------|-----------------------|
| GET /orders/demo               | HTTP 200             | Verificado            |
| Jaeger service-a               | Traza A → B → SQLite | 12 spans; 2 servicios |
| Logs por <code>trace_id</code> | Mismo ID que Jaeger  | Verificado            |
| Prometheus                     | Todos los targets UP | 4/4 UP                |
| Dashboard                      | Seis paneles         | Verificado            |

## 5. Evidencia visual de ejecución local

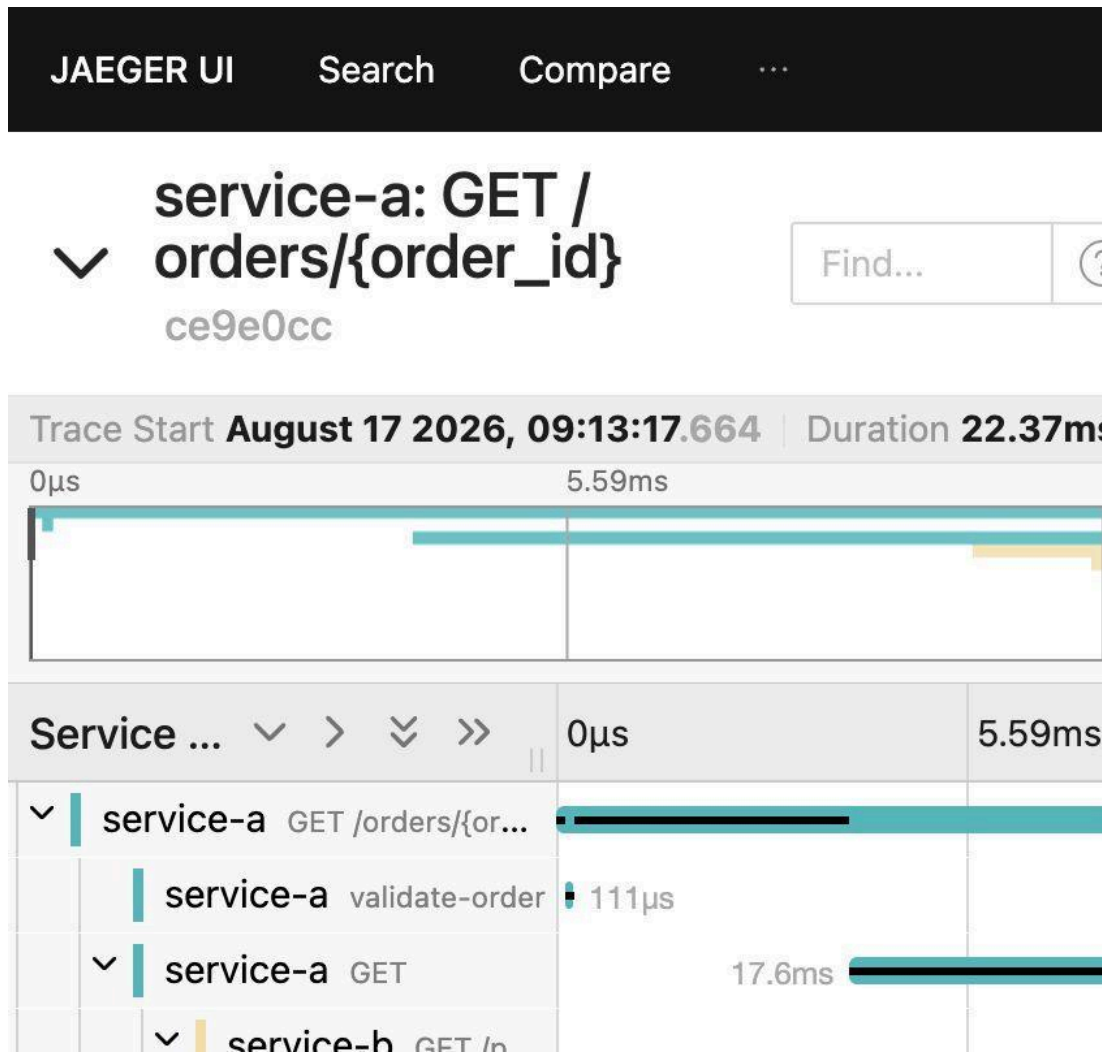


Figura 1. Jaeger: traza distribuida de service-a a service-b.

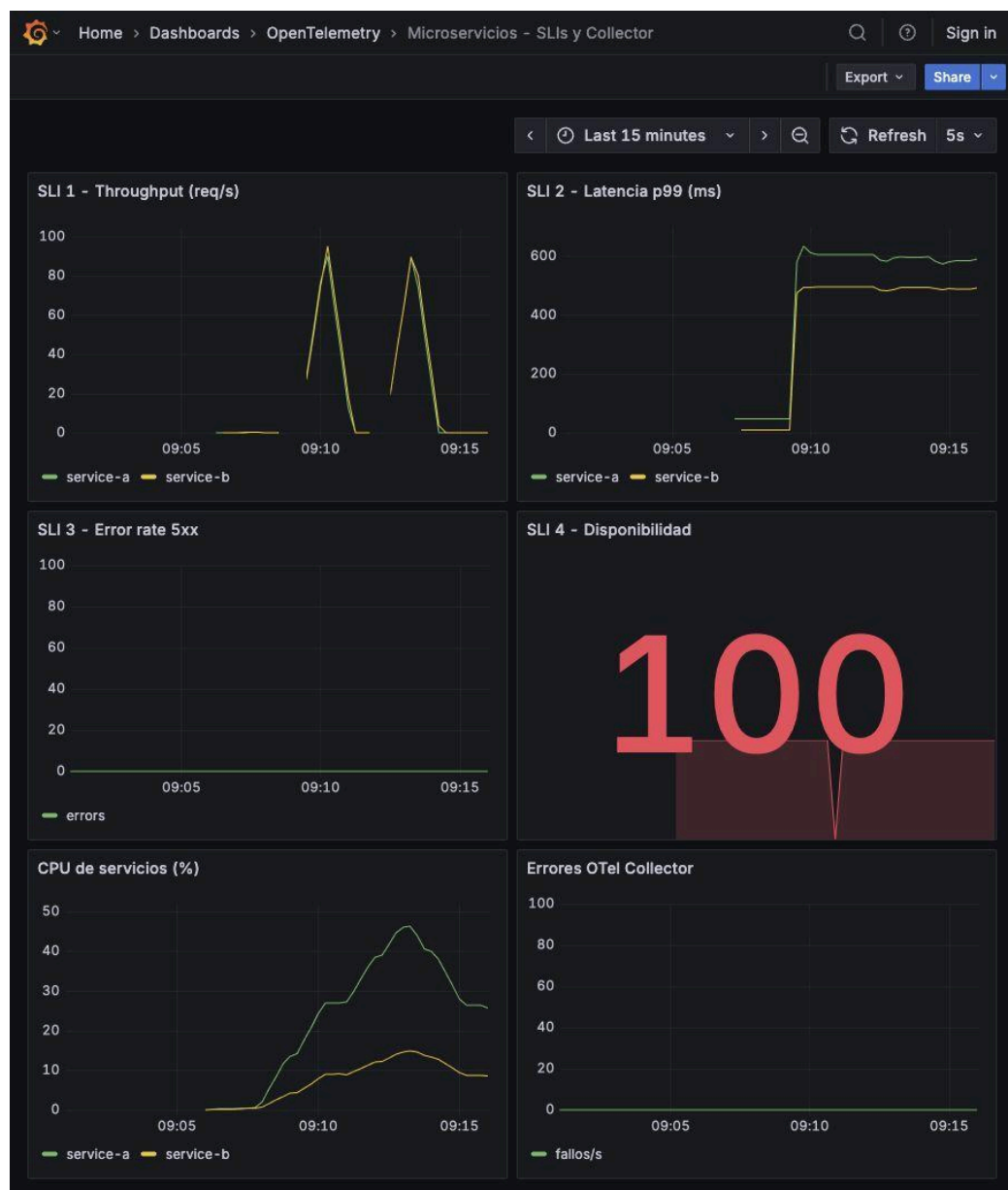


Figura 2. Grafana: dashboard con cuatro SLIs, CPU y errores del Collector.

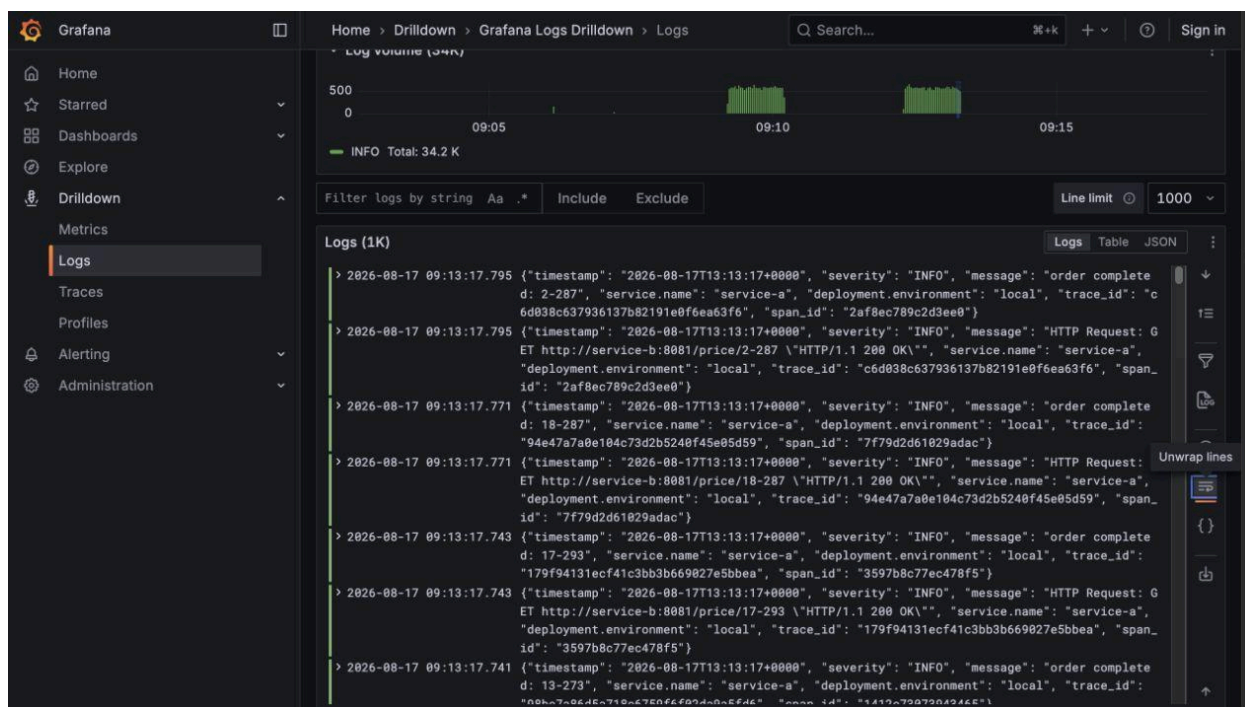


Figura 3. Grafana Logs Drilldown: líneas JSON con trace\_id y span\_id.

Las evidencias fueron obtenidas del stack Docker local con siete contenedores activos. Jaeger registró trazas de 12 spans con operaciones HTTP, validate-order, calculate-price, CREATE, INSERT y SELECT.

## 6. Análisis de overhead

Se ejecutaron dos condiciones con las mismas imágenes, 20 usuarios virtuales y 60 segundos. Baseline usó OTEL\_SDK\_DISABLED=true y la segunda condición habilitó OTel. k6 midió latencia y throughput; docker stats tomó 14 muestras de CPU y memoria conjunta por condición.

| Escenario           | p99 (ms)        | CPU (%)          | Memoria (MiB)  | req/s          |
|---------------------|-----------------|------------------|----------------|----------------|
| Sin instrumentación | 545.23          | 78.28            | 150.67         | 98.61          |
| Con OTel            | 559.52          | 88.39            | 145.55         | 93.49          |
| Diferencia          | +14.29 (+2.62%) | +10.11 (+12.91%) | -5.12 (-3.40%) | -5.12 (-5.19%) |

### Interpretación

En esta corrida, OTel añadió 14.29 ms al p99 y elevó el uso conjunto de CPU en 12.91%. El throughput bajó 5.19%. La memoria observada fue 5.12 MiB menor con OTel; esta variación debe interpretarse como ruido de asignación, caché y recolección de basura. No hubo errores HTTP en ninguna condición.

## Amenazas a la validez

Los resultados corresponden a una sola pareja de corridas en Docker Desktop. Cold starts, caché SQLite, recolección de basura y contención del host afectan la medición. Para una conclusión estadística se recomiendan tres repeticiones alternadas y comparación de medianas.

## 7. Decisiones de diseño

- Python facilita la lectura académica y dispone de auto-instrumentación madura para FastAPI, HTTPX y SQLite.
- OTLP desacopla las aplicaciones de los backends y permite cambiar destinos en el Collector.
- memory\_limiter y batch controlan presión de memoria y reducen el costo de exportación.
- Jaeger satisface la inspección de trazas; Prometheus y Grafana soportan SLIs; Loki habilita correlación local.
- El rol anónimo Editor de Grafana se usa solo en la demostración local para permitir Logs Drilldown sin login.

## 8. Reproducción

1. Ejecutar docker compose up --build -d.
2. Ejecutar scripts/smoke-test.sh para generar tráfico.
3. Validar los cuatro targets en Prometheus.
4. Abrir Jaeger y buscar service-a.
5. Abrir el dashboard Microservicios - SLIs y Collector en Grafana.
6. Ejecutar benchmark/run.sh y conservar los resultados.

## 9. Conclusión

El diseño integra los tres pilares y conserva el contexto entre servicios. La combinación de SLIs, trazas y logs correlacionados permite pasar de una alerta a la causa probable. El stack local, las evidencias y el benchmark fueron ejecutados; GKE y ECS permanecen como IaC validada porque requieren cuentas cloud autorizadas y pueden generar costos.

## Referencias

- OpenTelemetry Python SDK. <https://opentelemetry-python.readthedocs.io/>
- Jaeger Architecture. <https://www.jaegertracing.io/docs/architecture/>
- Grafana Trace Integration. <https://grafana.com/docs/grafana/latest/explore/trace-integration/>
- Grafana k6 Documentation. <https://grafana.com/docs/k6/>
- W3C Trace Context. <https://www.w3.org/TR/trace-context/>